

# Code Layout, Readability and Reusability

|               |                                                          |
|---------------|----------------------------------------------------------|
| Date          | 03 October 2026                                          |
| Team ID       | FITBUDDY-TEAM                                            |
| Project Name  | FitBuddy – AI Fitness Plan Generator using Gemini Models |
| Maximum Marks | 3 Marks                                                  |

## Project Structure

---

```
SDC-Project-main/  
|-- app/  
|   |-- main.py  
|   |-- routes.py  
|   |-- auth.py  
|   |-- schemas.py  
|   |-- models.py  
|   |-- database.py  
|   |-- gemini_generator.py  
|   |-- gemini_flash_generator.py  
|   |-- updated_plan.py  
|   |-- gemini_utils.py  
|-- templates/  
|   |-- base.html  
|   |-- index.html  
|   |-- login.html  
|   |-- new_plan.html  
|   |-- result.html  
|   |-- dashboard.html  
|   |-- all_users.html  
|-- static/  
|   |-- style.css  
|   |-- bmi.js  
|-- requirements.txt  
|-- .env.example  
|-- README.md
```

## Readability Practices

---

- Separate routing, authentication, database, schema and Gemini functionality into modules.
- Use descriptive function names such as `generate_workout_gemini()` and `update_workout_plan()`.
- Keep HTML templates separate from Python logic.
- Store API configuration outside source code.

## Reusability

---

The Gemini generation functions can be called by different routes, the database functions isolate persistence from presentation, and Jinja2 templates reuse common page structure through the base template.

## Ownership

---

Dinesh N – application structure; Afraar M – templates/CSS; Deepak B – AI modules; Vinesh D – validation/data modules.